Android Application Testing

Tanusree Nath
PhD Research Scholar
Department of Computer Science, BHU

Google scholar:
https://scholar.google.com/citations?user=h80-tYYAAAAJ&hl=en
LinkedIn: https://www.linkedin.com/in/tanusree-nath-5447291a7/

Outlines

- What is android application testing?
- Need for testing
- Manual vs automated testing
- Different types of manual tests
- Different types of Automated tests
- Unit testing using JUnit in Kotlin
- UI testing using Espresso in Kotlin
- Conclusion

What is testing?

- Process of checking whether an Android app works correctly, reliably, and provides a good user experience before and after it is released.
- Involves verifying things like functionality, performance, usability, security, and compatibility across different devices and Android versions.

Why do we need to test apps?

- Prevents crashes and bugs
- Improves user experience
- Ensures app works on multiple devices
- Protects user data

Types of testing

- Manual Testing: Humans test the app by using it
- Automated Testing: Scripts/tools automatically run tests

Manual tests

- Exploratory testing - Tester explores the app to find bugs
- Usability Testing - Checks how easy and intuitive the app is to use
- UI/UX Testing (visual aspects) - Layout, colors, fonts, alignment
- Ad-hoc Testing - Informal testing without predefined cases
- User Acceptance Testing (UAT) - Real users verify if the app meets requirements

Automated tests

- Unit testing – Test small code units (Tool: JUnit)
- Integration testing – Test interaction between different components
- Regression testing – Rerun tests after code changes to ensure stability
- API testing – check backend responses
- UI Automation testing – Test automated UI interactions (Tools: Selenium, Espresso)
- Performance testing – load, stress, speed testing

Unit testing

- Tests individual components (functions/classes)
- It does not require an android device
- Commonly used testing framework – JUnit

Unit Testing – using JUnit

Key features:

- simple and easy to use
- supports annotations
- provides assertion methods

Advantages:

- test functions without running full app
- very fast

Unit Testing – using JUnit

Key terminologies :

- Test case – A single test function
- Test suite – Collection of test cases
- Assertion – used to check actual vs expected output
- Annotation – Marks different test components

Unit Testing – using JUnit

What to test?

Ans – **A test case**

Example:

```
fun add(a: Int, b: Int): Int {
    return a + b
}
```

Unit Testing – using JUnit

How to verify results?

Ans – **Assertion methods**

Some common assertion methods:

- assertEquals(a, b) – check equality
- assertTrue(condition) – check true
- assertFalse(condition) – check false
- assertNull(value) – check null
- assertNotNull(value) – check not null

Unit Testing – using JUnit

How to mark different test components?

Ans - **Annotations**

Some common annotations:

- **@Test** – Marks a test method
- **@Before** – Runs before each test
- **@After** – Runs after each test

Unit Testing – using JUnit

Example – test **add(a,b)** using Junit

```
import org.junit.Test
import org.junit.Assert.*

class ExampleTest {

    @Test
    fun addition_isCorrect() {
        assertEquals(4, add(2, 2))
    }
}
```

UI Testing (Instrumentation Testing) – using Espresso

Used to test Android app user interfaces automatically by simulating real user actions.

Espresso simulates:

- Button clicks
- Typing texts
- Scrolling screens
- Verifying UI elements

UI Testing (Instrumentation Testing) – using Espresso

General structure of an Espresso test:

onView(MATCHER)

.perform(ACTION)

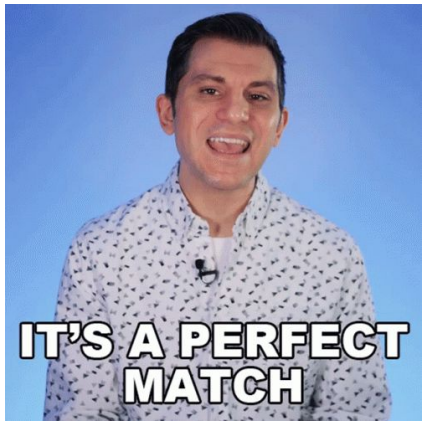
.check(ASSERTION)

Some useful espresso functions:

<u>1. Finders (used to find UI elements)</u>

- onView() - Selects a UI component from the screen.
  Example - **onView(withId(R.id.username))** finds element on the screen with id=R.id.username

UI Testing (Instrumentation Testing) – using Espresso



Some useful espresso functions (contd.):

<u>2. Matchers (used to match UI elements)</u>

- withID() – Finds UI element using a specific id.
  Example - **withId(R.id.loginBtn)** finds UI element with id=R.id.loginBtn
- withText() – Finds UI element by visible text.
  Example - **withText("Login")** finds UI element with the text “Login” on it
- **isDisplayed()** – Checks if element is visible.

UI Testing (Instrumentation Testing) – using Espresso

Some useful espresso functions (contd.):

<u>3. Performers (used to perform actions)</u>

- **.perform(action)** – Performs action on selected UI element.

Actions:

- **click()** – Simulates tap.
  Example – **perform(click())**
- **typeText()** – types text into input field
  Example - **perform(typeText("hello"))**
- **scrollTo()** – scrolls to element if not visible
  Example – **perform(scrollTo(element))**
- **closeSoftKeyboard()** – closes keyboard
  Example – **perform(closeSoftkeyboard())**

UI Testing (Instrumentation Testing) – using Espresso

Some useful espresso functions (contd.):

<u>4. Checker (used to verify results)</u>

- **check(assertion)**– Validates UI state.

matches() is a ViewAssertion function which checks whether a view satisfies a condition. It is often coupled with **matchers** like **withText()** and **isDisplayed()**
Examples–

- **check(matches(withText("Success")))**
- **check(matches(isDisplayed()))**

UI Testing (Instrumentation Testing) – using Espresso

```
@RunWith(AndroidJUnit4::class)                    //runs this test using JUnit4 test runner for Android
class LoginTest {                    //test class containing all test methods related to login functionality
   @get:Rule                                 //Launches MainActivity before each test runs
   val activityRule = ActivityTestRule(MainActivity::class.java)
   @Test
   fun testLogin() {                                    //simulates typing username
      onView(withId(R.id.username))
         .perform(typeText("testuser"), closeSoftKeyboard())
      onView(withId(R.id.password))                              //simulates typing password
         .perform(typeText("1234"), closeSoftKeyboard())
      onView(withId(R.id.loginBtn))                                  //simulates button click
         .perform(click())
      onView(withText("Login Successful"))                  //checks whether login is successful
         .check(matches(isDisplayed()))
   }
}
```

Conclusion

- Android testing ensures **quality, reliability, and user satisfaction.**
- Combines **manual testing** (exploratory, usability) and **automated testing** (unit, UI, integration).
- Tools like Espresso and JUnit enable **efficient and repeatable testing.**
- Early testing helps **detect bugs, reduce costs, and improve performance.**
- UI testing frameworks such as Espresso simulate **real user interactions.**
- A well-tested app leads to **better user experience and higher trust.**

Key-takeaway

“Effective testing is essential for building robust, scalable, and user-friendly Android applications.”